

CAS 1102 Project 1: Omok++

Due date 1 : Sept 28, 2026 Noon KST (Game Infrastructure)

Due date 2: Oct 19, 2026 Noon KST (Game Agent)

General instructions & precautions:

- The project is worth 25 pts towards your final score of 100 pts.
- **You may work in teams of sizes 1-4.** Since teams with more members have an advantage, teams of size 3 will receive an adjustment of +0.5 pts, 2 will receive an adjustment of +1 pts, and 1 will receive an adjustment of +1.5 pts. There are also other benefits of smaller teams involving extra credit. You are responsible for forming your own team; the teaching staff will not be involved in your team dynamics and everyone in the same team will get the same score. However, we will ask you to report the role of each member. You must keep your team for both parts of the projects, but you may change teams for Project 2. Even if a member of your team drops out of class, we will grade based on the team size reported at Due Date 1. You may not add additional members.
- You may freely collaborate with members of other teams, or utilize other sources, but you need to clearly list them out and how you utilized them. Code from other teams or other outside sources should not be utilized verbatim; doing so will be considered cheating and will result in an F for this class.
- **LLM usage is not recommended for designing and building the game infrastructure, while we expect you to utilize LLMs when designing the game agents.** We will not strictly ban the use of LLMs as long as you are clear about how you utilized LLMs and which parts of the code were written by LLMs (through comments on the code) as well as explicit reporting. You must be able to explain/reproduce your own code. LLMs should be a tool to promote your learning, not something to get in the way of learning. My recommended use of LLMs for the game infrastructure is to (1) plan out the general structure by hand, (2) try your best to perform an implementation without the help of an LLM as much as you can, and if you wish, (3) utilize LLMs for feedback or to create a separate code to see if there are any surprises when compared to your code.

- If there is evidence or concern that your team member cheating (e.g., unreported use of LLMs, including malicious code, etc.), please report to us before the assignment due date or as soon as you become aware of the fact. We will disregard that team member when computing the grades. If you do not report your team member's cheating before it is discovered, the entire team will be regarded as having cheat in the class and will all receive an F for the class.
- If your code fails to run or compile, the TAs will not grade the project and return a '0' pt score for the relevant section (either Game Infrastructure or Game Agent), unless otherwise specified. Please double-check that your submission compiles and executes. If there is any dispute, you may show us that you can download the submission and run it on your machine. Please note that your assignments will be graded in the absence of an internet connection. In other words, your code cannot call API or utilize sources on the web during execution. If your code fails to execute for this reason, it will count be considered as 'failing to run'). Due to the size of the class, we cannot dig into your code and debug each one of your code.
- Your code must only depend on the 'standard libraries' or libraries that are widely used (typically installable through `pip` without complicated installation process). If unsure, please ask on the course Slack channel `prj1-omok`. If your code has non-standard dependencies, we cannot go through the effort of installing the dependencies for every project, and your project will not be graded and will receive a 0.
- Being up to 24 hours late would penalize your score by 20%. The next 24 hours will increase the penalty by an additional 40% for a total of 60%. No submissions will be accepted past the 48 hours after the deadline mark. Extensions will not be granted less than 48 hours before the deadline, except under extraordinary unforeseeable circumstances.
- The assignment includes chances for various extra credit, but be aware that they are difficult and potentially time consuming. Many will require you to learn significant amount of materials beyond the class. Moreover, to account for the team sizes, we will 'distribute' the extra credit equally among the team members. e.g., each member of a team of size 4 that earned 10 extra credit pts will receive 2.5 pts, whereas the solo member of a team of size 1 that earned 10 extra credit pts will receive 10 pts.
- Please pay careful attention to the submission instructions. If you fail to follow the submission instructions (e.g., wrong file name or extension (e.g., pdf)) when specifically requested, we will not grade that part of the submission and you will receive 0 pts for the specific part(s).
- For any questions about the project, use the course Slack channel: `prj1-omok`. For each bug report, the first team to report will be awarded 0.5 pts.
- Do **not** upload your solutions to the public domain.

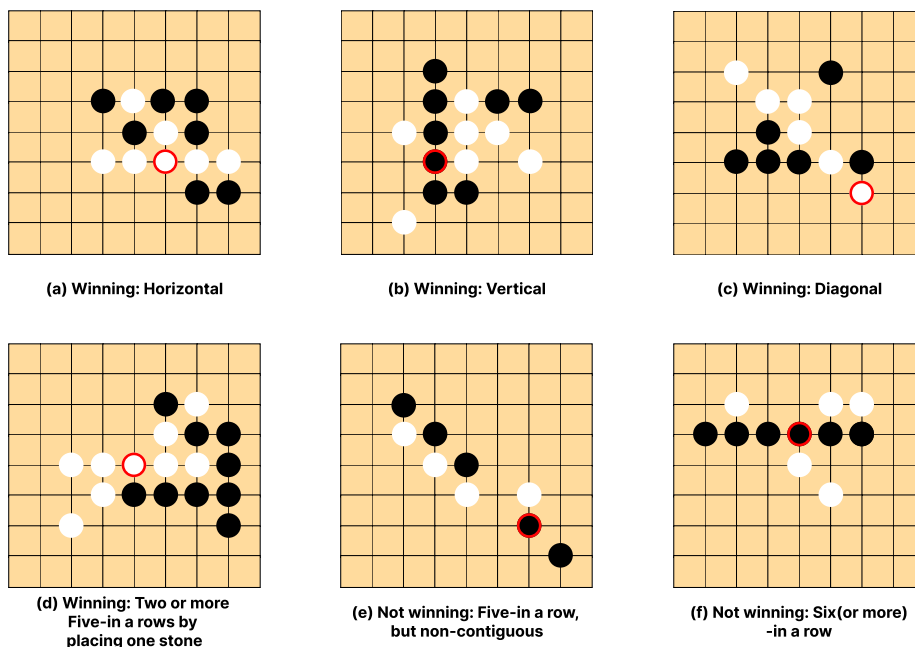


Figure 1: Examples of winning positions (a-d) and non-winning positions (e (non-contiguous), f (6 stones in a row)). The lastly added stone has a thick red boundary.

Project Description

In this project, you will be implementing a variant of the game called *omok* (also known as *five-in-a-row*, *gomoku*, or *wuziqi*). For those of you who do not know this game already, omok is a two-player game that you play with black and white stones on a 19-by-19 go board. The two players take turn, starting with black. then white, then, black again, then white again, and so on and so forth, until one of the the two players win by getting five stones in a single contiguous row (horizontal, vertical, and diagonal are all allowed).

Please note that although there are multiple variants of the game, we will consider the most basic version of the game in which players can place their stones on *any* of the positions that does not already have a stone, and one must get exactly five stones in a row (i.e., if you end up with 6, 7, 8, or 9 stones in a row, that does not count). Please see Figure 1 for example winning and non-winning positions.

If the board ends up being completely filled up without either player winning, the game is considered a *draw*.

In this project, we will implement *omok++*, a variant of the game in which a player can ‘take’ the other player’s stones off the board. The added rule is simple:

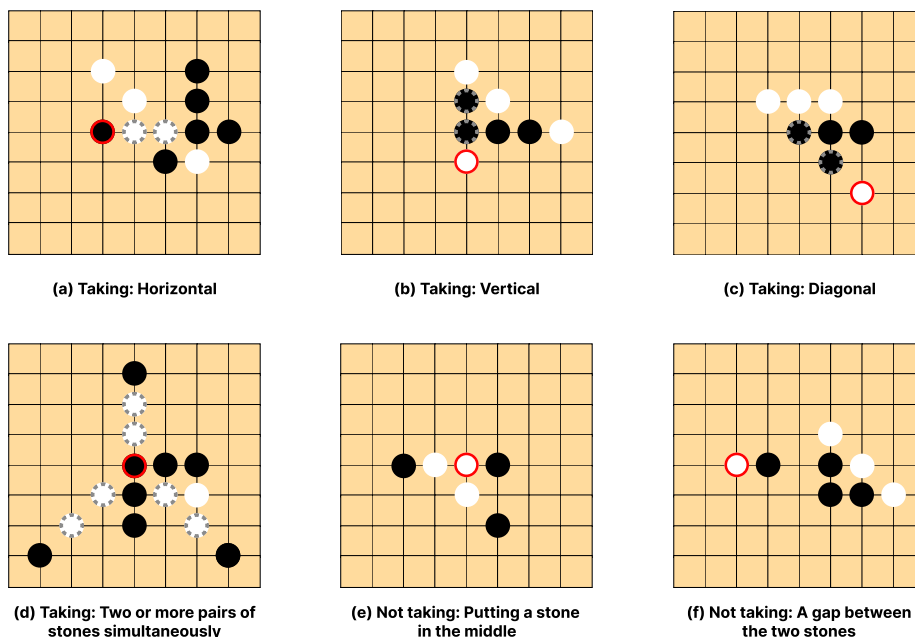


Figure 2: Examples of taking positions (a-d) and non-taking positions (e (the new stone ‘filled in’ the space between the opponent stones), f (6 stones in a row)). The lastly added stone has a thick red boundary and the removed stones as a result are surrounded with dotted boundaries.

If the stone you just placed on the board leaves *exactly two contiguous opponent stones* surrounded by your stones on both ends, one of which is the stone you just placed, the two surrounded opponent stones are taken off the board. (horizontal, vertical, and diagonal are all valid ways to remove opponent stones).

Examples of taking and non-taking positions are shown in Figure 2. Please note that after taking the stones, the positions the stones used to occupy are now vacant again and either player may put their stones there in later turns.

Note that the opponent can win after your taking move: if you take one of the stones forming a 6-in-a-row (contiguous) so that the taking move makes it a 5-in-a-row (contiguous). If both players have 5-in-a-row simultaneously as a result after a move, it is a tie.

This mechanism of being able to take off leads to an interesting space of new strategies, which you would likely understand better by playing the game out a couple of times. I highly recommend playing this game out with your friends couple of times to familiarize yourself!

1 Game Infrastructure (10 pts + 4 pts (extra))

First, we will set up the game infrastructure on which you and other AI agents can play the game of *omok++*, as described above.

To guide your implementation, we provide the implementations of two simple AI bots for playing this game: **RandomPlayer** (in `randomplayer.py` and **SimplePlayer** (in `simpleplayer.py`). These bots will be fairly easy to beat, but will tell you how the **Players** are structured. You should be able to test your implementation against these bots.

Couple requirements for the implementation:

- The `omok.py` file should serve as the entry point to the program, one must run the system by typing `python omok.py [player-1] [player-2] [time-limit]` (may be `python3` or something else, depending on the machine) into the terminal (or executing with 3 additional parameters). For players, you have the following codes: 0: human player, 1: **RandomPlayer** (provided), 2: **SimplePlayer**, and 3: the agent that you would create during the second segment of the class (See 2. Game Agent) – this part is not expected to function for the first submission deadline. For the `time-limit`, you are giving an integer number of minutes (allotted for the player throughout the entire game, not per turn). Giving `-1` indicates that there is no time limit. e.g., `python omok.py 0 0 -1` would initiate a game with no time limit between two human players and `omok.py 1 3 1` would initiate a game between your agent and the provided **RandomPlayer** with a 1 minute time limit (hint: there are two widely used methods for reading in arguments: `argv` in `sys` or `argparse`. You may want to read into these.).
- All types of players are represented with the base class **Player**. Any player that you implement (including **HumanPlayer**, which must handle the interaction with the human player) must directly inherit from this class. You will also notice that the provided example classes directly inherit from the **Player** class (You may have intermediate inheritance as well, if you feel there is a need for one (e.g., **AutomaticPlayer** representing the various game agents). Each of the players should have a `take_turn()` method, which is passed (1) `board`, which represents the board state as an integer array of dimension 19-by-19, with `-1`'s representing empty positions where one can play on, `0`'s representing the positions of the first player (black), and `1`'s representing the positions of the second player (white); (2) `time` representing the remaining time for the player, given in milliseconds (ms). The player class should NOT directly modify the `board` provided, but rather return a pair of integers representing the location of the next move. The indices on the board are 0-based and the upper left corner of the board is `board[0][0]`, `board[1][0]` represents the leftmost position on the second row from the top, and `board[0][1]` represents the topmost position on the second column from the left.
- For the **HumanPlayer**, your `take_turn()` method should have a mechanism

of confirming that the played at a valid position, given in the format `n,m` (e.g., `13,4`; no whitespaces between); if the human player played in an invalid position (e.g., invalid input, out-of-bound values, occupied position), the player should be asked to re-enter the move (hint: if you have never worked with user inputs before, you may want to read into `input()`). Moreover, it should display the time remaining for the human player at the beginning of their turn. For the non-extra credit, you should not need to perform any cut-off or take any actions about out-of-time loss on the `HumanPlayer` class; if the turn is taken and it exceeded the allowed time, the `GameManager` (to be described in the next bullet) should be the one issuing the out-of-time game over.

- There should be a `GameManager` class that manages the two players in the game. This class should be the one instantiating the two players and managing the alternation of the turns between the two players by calling on the `take_turn()` function of the two player instances. Once the player turn is completed and the player move has been obtained, it should update the board, display the board (on the terminal (for the non-extra credit)) (these two functionalities may be performed via a separate `Board` class), and check the victory conditions. For the non-extra credit, it should clearly print out the winner of the game and terminate.
- While we do recommend having a separate `Board` class that manages the board (e.g., placing the stone, returning the array form of the board, printing the board on the screen/turning the board into a string, etc.), it is optional to have one and the design – if you decide to have one – is completely up to you.
- Your implementation of the game infrastructure should work with any reasonable implementation of the player agents that follow the requirements set here. Also, your implementation of `HumanPlayer` should be able to work with any reasonable implementation of the game infrastructure that follow the requirements set here. While you are allowed to test your code against each other (you need to acknowledge the other team clearly), be aware not to end up leaving your code with the other team. If the two codes have significant overlap, we will consider both teams as having cheated, and all members of both teams will receive an ‘F’ for the course. If you fail to acknowledge the other team, all members of your team will be considered as having cheat in the class.

Please note that we do not recommend the usage of LLMs or other types of AI is **not recommended** for this part of the project for your learning. Even if you decide to use it, we hope you use it in a way that does not interfere with your learning to induce “productive struggle.”

Submission Instructions (Important!): Please collect all the files and submit as a single zip file, named: `omok-infrastructure-[team-name].zip` (e.g., `omok-infrastructure-GodOfOmok.zip`). At the base directory, include a file

named `README.txt` that lists (1) the members of the team, (2) compilation or running instructions (including any packages that may require installation) (may be as simple as the terminal commands if there is nothing special and do **NOT** leave this section blank), (3) a declaration of LLM or AI usage (details about how you used LLM or AI and what parts of the code has been written by LLM or AI, and which LLM/AI you used (including the plan, if using a paid plan)) (*If you did not use LLM or AI, please write “I did not use AI for this project.” and do **NOT** leave this section blank*), (4) acknowledgment of any resources you utilized for the project, including sources on the web, other friends in the class, etc. (*If you have nothing to acknowledge, please write “N/A” and do **NOT** leave this section blank*), (5) which extra credits you are submitting for (optional), and (6) any other notes you wish to include (optional). Also in the base directory, include the `class-diagram.pdf` from 1.1. Then, make a subdirectory named `omok-game` and include all the files inside that directory. Moreover, if you used LLMs, include a directory called `llm-usage` and include all prompts and responses you obtained in the process. If you attempt the extra credit, you should make a separate directory named `omok-game-extra` at the same level as `omok-game`. Please refer to the extra credit portion (1.4) for more details.

Please be aware that if your submission does not meet the aforementioned submission instructions (e.g., missing the declaration of LLM or AI usage), we will **NOT** grade your work and you will receive a 0 for the submission of this part.

1.1 Designing the Classes (2 pts)

Draw a rough schema of the classes, member variables, and their methods.

Represent each of the classes as a two-slotted box with its name designated above. The top part of this box should list out the member variables (including the type (roughly, since Python has flexible types), private/public, instance/-class/static) and a short description of what the variable depicts. The lower part of this box should include the methods (instance/class/static, input and output types and information) and a short description of what the method is supposed to do. If the class is intended to be an *abstract* class, have a dotted boundary instead of a solid one.

For inheritance, please draw an solid line with an arrow from the parent class to the child class. Moreover, if a class is instantiated as a member variable, draw a dotted line connecting the class with the member variable of interest. If a class is instantiated within a method, draw a dotted line connecting the class and the method it is instantiated in.

The schema may be drawn either by hand or by using software, but should **NOT** be generated by LLM or AI; we will not award any points if you do use LLM or AI (+ if you do not report the usage, then it would be cheating). If the schema is hand-drawn, the text must be clearly legible.

Submission Instructions: Please submit your solution to this problem as the

file `class-diagram.pdf`. The file should be a single "PDF" file.

How this part will be graded: If you fail to meet any 1 of the requirements listed in the descriptions given in the project description, description of the game infrastructure, or this problem, we will award half the points. If there are more than one requirements that the submission fails to meet, you will receive no points.

1.2 Implementing The Game Mechanism (5 pts)

Implement the game mechanism, including the `omok.py` as well as the `GameManager` and any other implementation required to get the provided player agents (and any other agents meeting the requirements) to play against each other. Moreover, set up the required abstract classes (i.e., `Player` and any other ones). The implementation should follow your design from 1.1 (you may go back to make the needed modifications).

How this part will be graded: If your code fails to execute or run, or if it has any critical issues that renders the system unusable (e.g., crashing a significant fraction of the times like 1 in 10, does not print the board at all), you will not receive any points for this part. If your code has playable major bugs (e.g., captures not being handled, occasionally wrong victory detections, not capturing the tie conditions, etc.), we will award 1 pt; if your code has minor bugs (e.g., specific very infrequent conditions causing crashes, etc.), we will award 2 pts.

1.3 The HumanPlayer Class (3 pts)

Now, implement the `HumanPlayer` class. After this is implemented, you should be able to play against bots or against another player (on the same machine).

How this part will be graded: If your code fails to execute or run, or if it has any critical issues that renders the system unusable, you will not receive any points for this part. If your code has playable major bugs, we will award 0.5 pt; if your code has minor bugs, we will award 1 pt.

1.4 Extra Credit (4 pts)

Please note that the extra credit problems are intended to be hard and may require you to self-study significant amount of materials. I would not recommend trying unless you are willing to spend a lot of time. We will not offer help on the extra credit.

If the base version does not work or had any bugs or had deductions, we will **NOT** grade your extra credit. Make sure your base game works without issues before attempting the extra credit. Moreover, you either 'get the points' or 'do not get the points'.

Submission Instructions: If you attempt the extra credit, you must submit the code in a separate directory from the base code. Please name the directory

`omok-game-extra`. Within this directory, include a separate `README-extra.txt` any compilation or running instructions (including any packages that may require installation). Any declarations or acknowledgment should have already been included in the earlier `README.txt`.

1.4.1 Multiple Sessions (1 pt)

While the base version only worked for a single session of the game, you can make the game work over multiple sessions.

Modify the program so that you can have multiple games. Things you need to support:

- Selecting your opponent from a list (`HumanPlayer`, `RandomPlayer`, and `SimplePlayer`)
- Once a game ends, your victory/loss counts show up and you can either play another opponent or exit.
- You should be able to play the game as many times as you wish.

Please note that you should not modify any of the `Player`-related code.

1.4.2 GUI (1 pt)

(Only if you have the multiple sessions properly working) Now, make the game visual. Make the game display the game board, not inside the terminal. The user should be able to click on the position they want to place their stone. Any options for selection must also be selectable through clicking on the interface.

1.4.3 Timeout (2 pts)

(Only if you have both of the previous extra credit properly working) *Without modifying any of the `Player` classes or its children*, allow your `GameManager` to monitor the children classes and upon running out of time, terminate the `Player`'s turn and immediately declare a loss due to running out of time. (If you modify the `Player` code or any of its children classes, then we will not give you any extra credit)

2 Game Agent (15 pts + 8 pts (extra))

Now, you will create your own intelligent agent for playing the game.

Make your own `SmartPlayer[TeamName]` class (e.g., `SmartPlayerGodOfOmok`) that is capable of playing the game of `omok++` best.

Submission Instructions (Important!): Please collect all the files and submit as a single `zip` file, named: `omok-agent-[team-name].zip` (e.g., `omok-agent-GodOfOmok.zip`). At the base directory, include a file named `README.txt` that lists (1) the members of the team, (2) compilation or running instructions (including any packages

that may require installation) (may be as simple as the terminal commands if there is nothing special and do **NOT** leave this section blank), (3) a declaration of LLM or AI usage (details about how you used LLM or AI and what parts of the code has been written by LLM or AI, and which LLM/AI you used (including the plan, if using a paid plan)) (*If you did not use LLM or AI, please write “I did not use AI for this project.” and do **NOT** leave this section blank*), (4) acknowledgment of any resources you utilized for the project, including sources on the web, other friends in the class, etc. (*If you have nothing to acknowledge, please write “N/A” and do **NOT** leave this section blank*), and (5) any other notes you wish to include (optional). Make a subdirectory named **omok-agent** and include all the files inside that directory; please submit all files that your agent inherits from (including the **Player**). To avoid name clashes, please add your team name at the end of all the classes included this way (e.g., **Player** → **PlayerGodOfOmok**), including any intermediate classes. Name the files in the same way by adding your team names at the end. Moreover, if you used LLMs, include a directory called **llm-usage** and include all prompts and responses you obtained in the process.

How this part will be graded: If your code fails to run, you will not receive any points. You get points for defeating the various agents. The provided time limit is 1 minute throughout the game (again, this is not 1 minute per turn). For 1, 2, 3 below, ‘beat’ is defined as being able to strictly win (not draw) the agents at least 5 out of 8 games (4 as black and 4 as white).

1. If your agent is able to beat **RandomPlayer**, you will obtain 2 pts and move on to 2.
2. If your agent is able to beat **SimplePlayer**, you will obtain 3 pts and move on to 3.
3. If your agent is able to beat the two agents the TAs built (using LLMs, but with limited time), you will obtain 2 pts for each TA agent your agent is able to beat. If your agent is able to beat both of the TA agents, move on to 4. (The TA agents are not released)
4. We will have all agents that have made it this far play against each other 8 times (4 as black and 4 as white). ‘Winning’ is defined as having won strictly more games than the opponent. For each ‘win’, you get 2 coins, and for each ‘draw’, you get 1 coin (the opponent does too). The bottom 50% of those who qualified for 4 will not receive any more pts. The next 30% will receive 3 pts. The top 20% will receive 6 pts, which would be the full score. On top of this, if there are at least 20 qualifying teams, the top team will obtain 8 extra credit pts and the runner-up team will obtain 3 extra credit pts. If there are between 10 (inclusive) and 20 (exclusive) qualifying teams, the top team will obtain 4 extra credit pts.

Please note that if your agent crashes in any of the 8 games, we will consider your agent as having lost against the opponent agent.